SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Database Management Systems
COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO2: *Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems.* (BL3, BL6)

Activity Tool : Analytical Problem Solving (High)
Assessment Tool Weightage: 40%

QUESTIONS		
Q.No	**Question**	**Bloom's Level**
1	Given a set of relations and sample data, write SQL queries to retrieve specific information using joins and subqueries, and explain the logic used. Bloom's Level: BL3 – Apply	BL4 – Analyze
2	Using relational algebra, formulate expressions to solve complex queries such as retrieving records that satisfy multiple conditions across relations. Bloom's Level: BL4 – Analyse	BL5 – Evaluate
3	Given a real-world scenario (e.g., banking or student system), design a relational schema and construct appropriate SQL queries for data retrieval and updates. Bloom's Level: BL6 – Create	BL6 – Create
4	Analyse a given SQL query with nested subqueries and predict its output, explaining each step of execution. Bloom's Level: BL4 – Analyze	BL6 – Create
5	Compare different SQL approaches (joins vs subqueries) for solving the same problem and evaluate their efficiency. Bloom's Level: BL5 – Evaluate	BL4 – Analyze
6	Design and implement views for a database system to provide restricted data access, and justify their use for security and abstraction. Bloom's Level: BL6 – Create	BL5 – Evaluate
7	Given a database schema, write relational algebra expressions and convert them into equivalent SQL queries, analyzing differences in representation. Bloom's Level: BL4 – Analyze	BL4 – Analyze

8	Optimize a set of inefficient SQL queries by restructuring them using joins, indexing concepts, or views, and evaluate performance improvements. Bloom's Level: BL5 – Evaluate	BL5 – Evaluate
9	Develop a complex SQL query involving grouping, aggregation, and nested queries to solve a real-world problem. Bloom's Level: BL6 – Create	BL6 – Create
10	Given multiple relations, analyze the query requirements and decide whether to use views, joins, or subqueries, justifying your choice with reasoning. Bloom's Level: BL5 – Evaluate	BL5 – Evaluate

---

Code of Conduct:

I **K. Sampath Chaitanya (192572006)** *certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

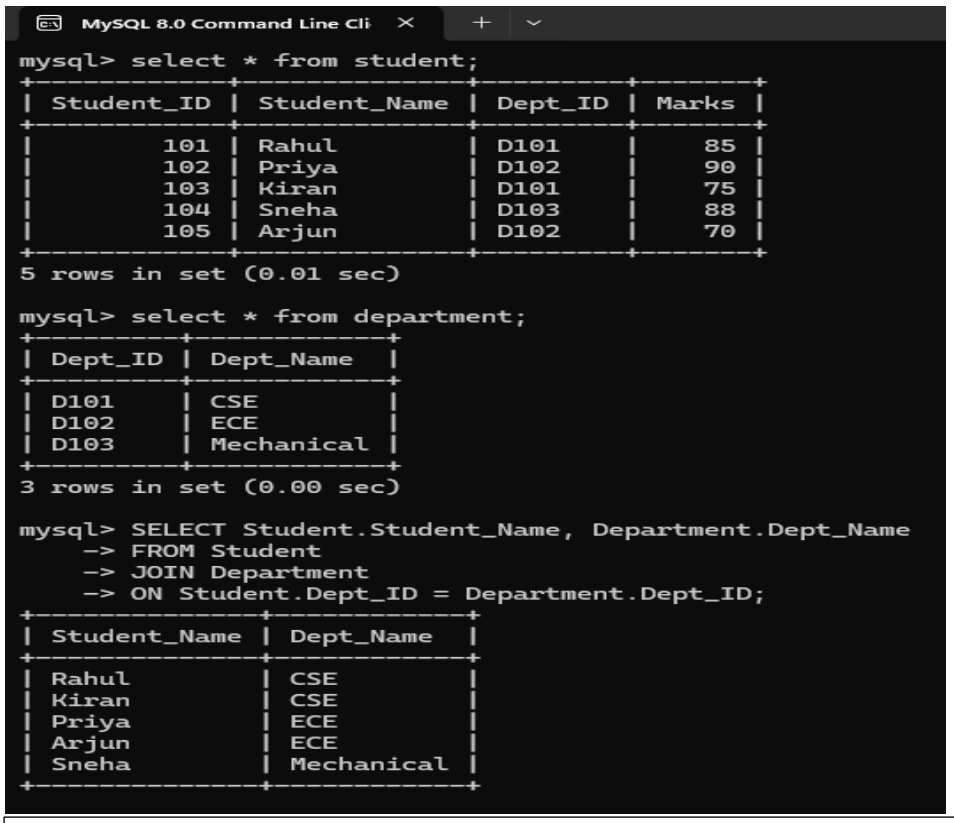
Signature of the student:

RUBRICS FOR CALCULATION:

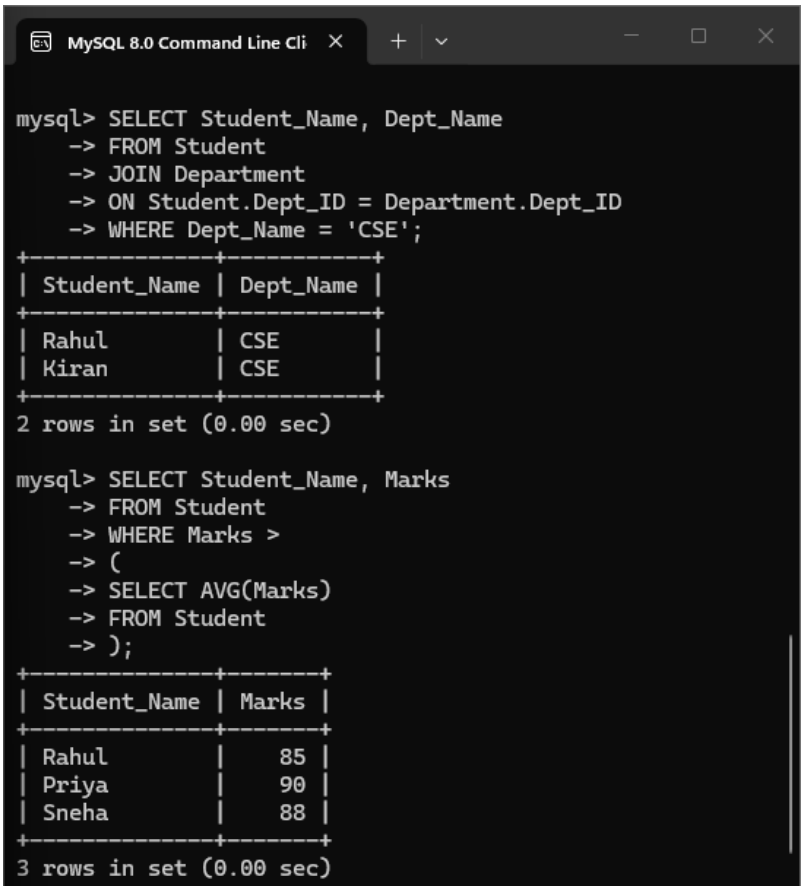
Criteria	**Level 4 (Exemplary)**	**Level 3 (Proficient)**	**Level 2 (Developing)**	**Level 1 (Beginning)**
Problem Understanding	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterpret s or unable to understand problem
Method Selection	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process

Accuracy of Calculation	All calculations accurate;	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
	correct final answer			
Interpretation of Results	Clearly interprets results with units and engineering relevance	Basic interpretatio n with minor omissions	Limited or unclear interpretation	No interpretatio n

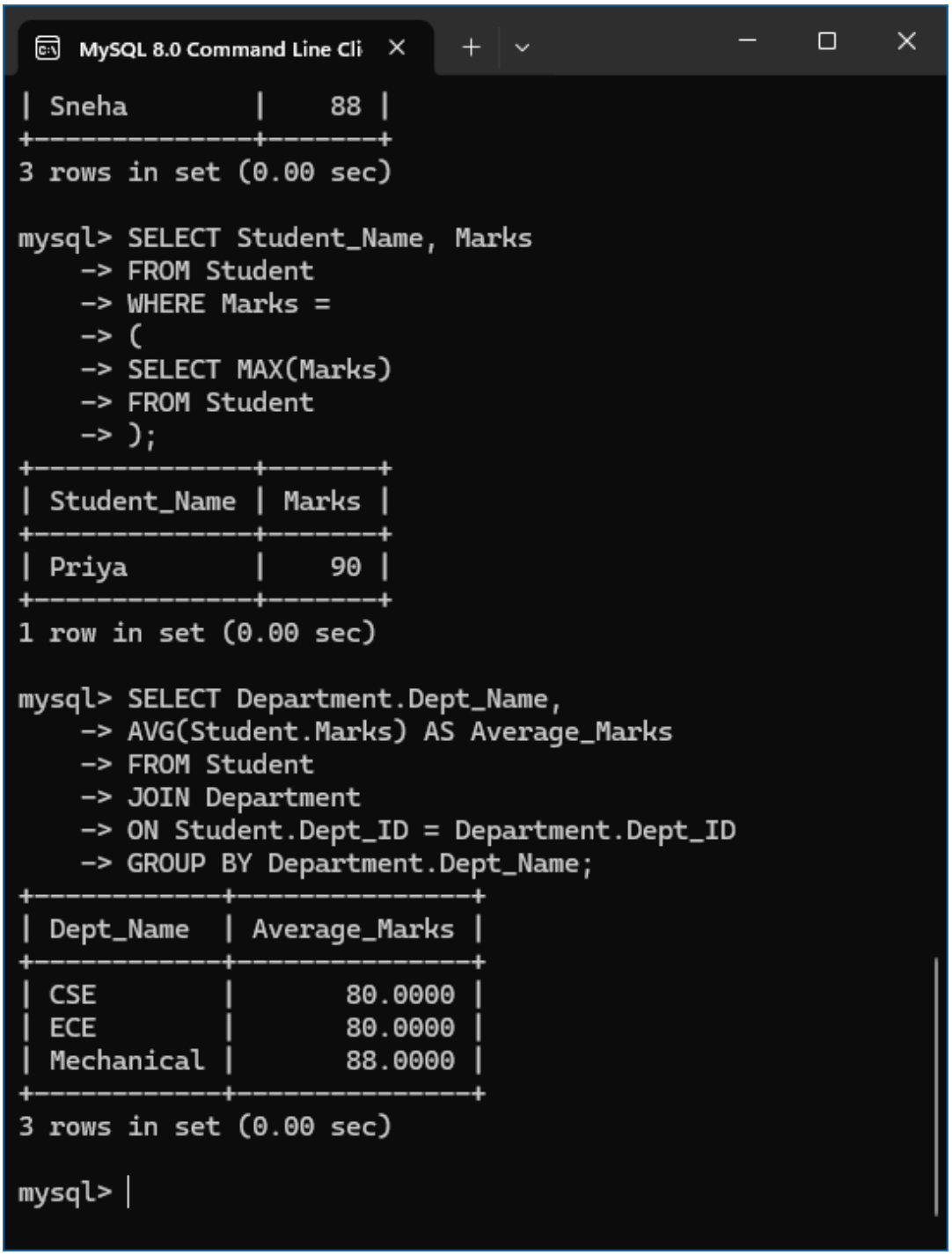
1… Join Query



INNER JOIN with Condition and subquery



Subquery with Maximum Marks and join plus aggregation



}

Justification:
Joins are used to combine data from multiple related tables using a common attribute, while subqueries retrieve intermediate results that are used by the main query. This approach helps in extracting accurate and meaningful information from relational databases.

1) Using relational algebra, formulate expressions to solve complex queries such as retrieving records that satisfy multiple conditions across relations.

Given Relations

Student_ID	Name	Dept_ID	Marks
101	Rahul	D101	85
102	Priya	D102	90
103	Kiran	D101	75
104	Sneha	D103	88
105	Arjun	D102	70

Department

Dept_ID	Dept_Name
D101	CSE
D102	ECE
D103	Mechanical

Query 1
Retrieve the names of students who belong to the CSE department.
Relational Algebra

$$\pi_{Name}(\sigma_{Dept\_Name='CSE'}(Student \bowtie Department))$$

Logic

- Join Student and Department using Dept_ID.
- Select only rows where Dept_Name = 'CSE'.
- Project only the Name attribute.

Query 2
Retrieve students who scored more than 80 marks.

Relational Algebra

$$\pi_{Name,Marks}(\sigma_{Marks>80}(Student))$$

Logic
Select students with marks greater than 80.

Display only Name and Marks.

Query 3
Retrieve the names of students from the ECE department who scored above 75.
Relational Algebra

$$\pi_{Name}(\sigma_{Dept\_Name='ECE' \wedge Marks>75}(Student \bowtie Department))$$

Logic
Join Student and Department.

Apply two conditions:

Department = ECE

Marks > 75

Display only student names.

Query 4
Retrieve all department names having students with marks greater than 85.
Relational Algebra

$$\pi_{Dept\_Name}(\sigma_{Marks>85}(Student \bowtie Department))$$

Logic

- Join both tables.
- Select students with marks above 85.
- Display department names.

Query 5
Retrieve names and department names of students whose marks are between 80 and 90.
Relational Algebra

$$\pi_{Name,Dept\_Name}(\sigma_{Marks\geq 80 \wedge Marks\leq 90}(Student \bowtie Department))$$

Logic
Join Student and Department.

Select students whose marks are between 80 and 90.

Display student name and department name.

Query 6
Retrieve all students who belong to either the CSE or ECE department.
Relational Algebra

$$\pi_{Name}(\sigma_{Dept\_Name='CSE' \vee Dept\_Name='ECE'}(Student \bowtie Department))$$

Logic

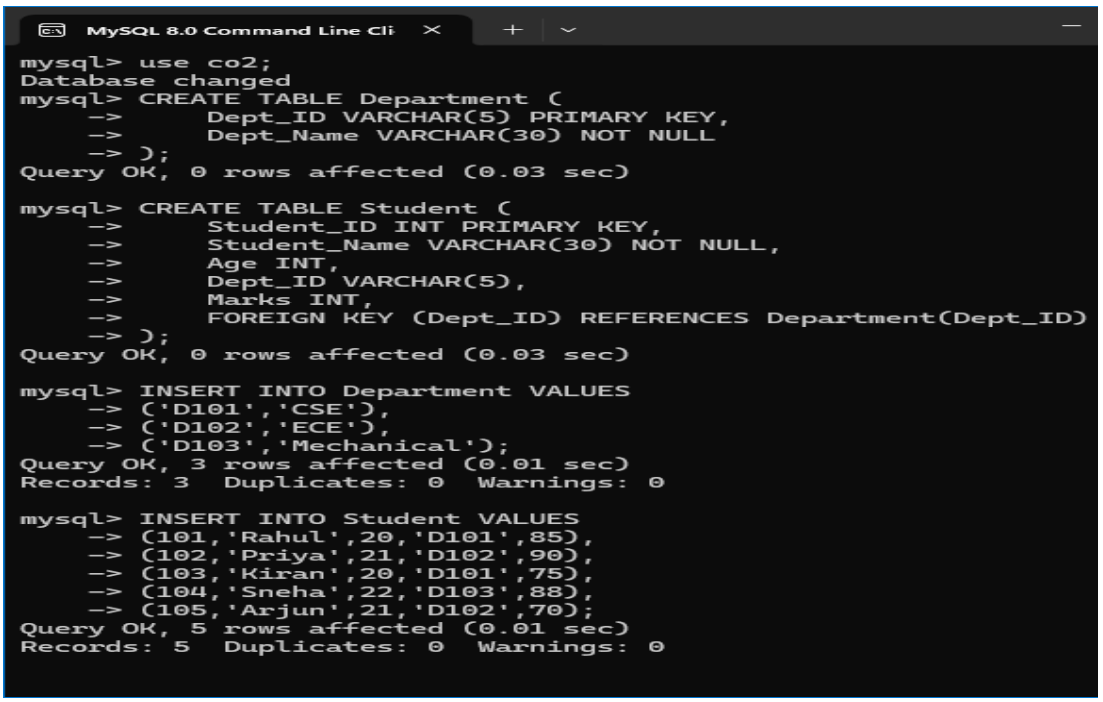
- Join the relations.
- Select students from CSE or ECE.
- Display only their names.

Important Relational Algebra Symbols (Exam)

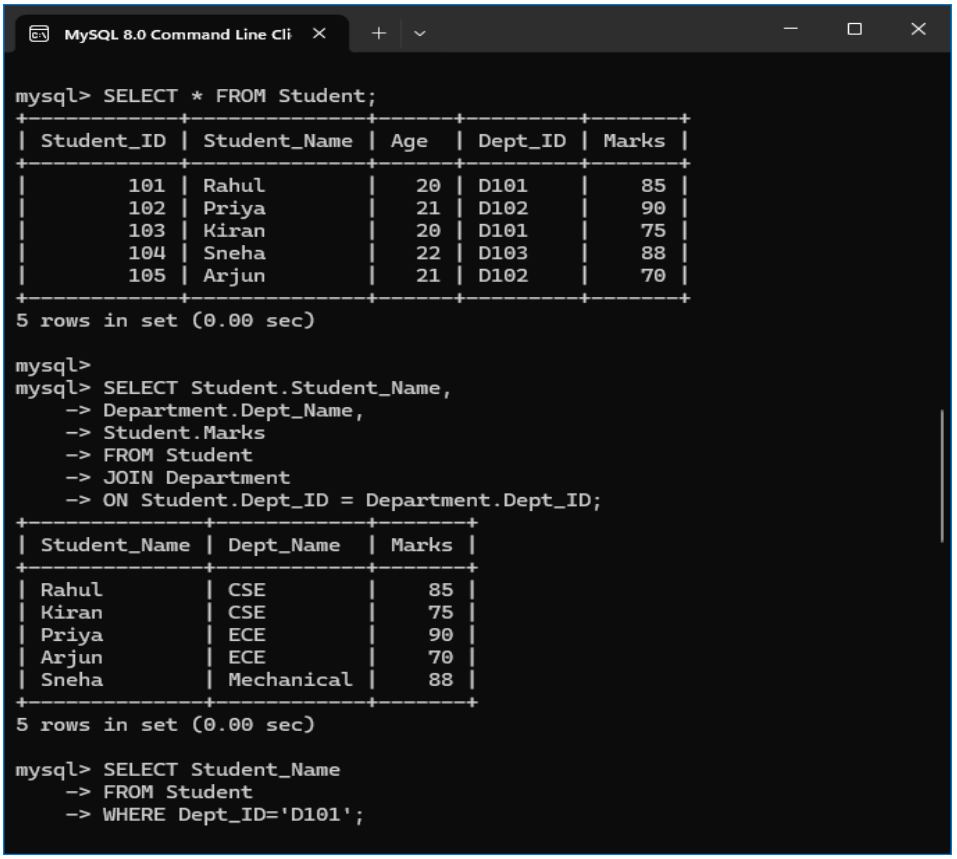
Symbol	Meaning
σ (Sigma)	Selection (rows)
π (Pi)	Projection (columns)
⋈ (Join)	Join two relations
×	Cartesian Product
∪	Union
∩	Intersection
–	Set Difference
ρ	Rename

Justification:
Relational algebra provides a mathematical foundation for querying databases using operations such as selection, projection, and join. It helps analyze how data is processed before implementing the query in SQL.

2) Given a real-world scenario (e.g., banking or student system), design a relational schema and construct appropriate SQL queries for data retrieval and updates.



SQL Queries - Retrieve students with department names, Retrieve CSE students

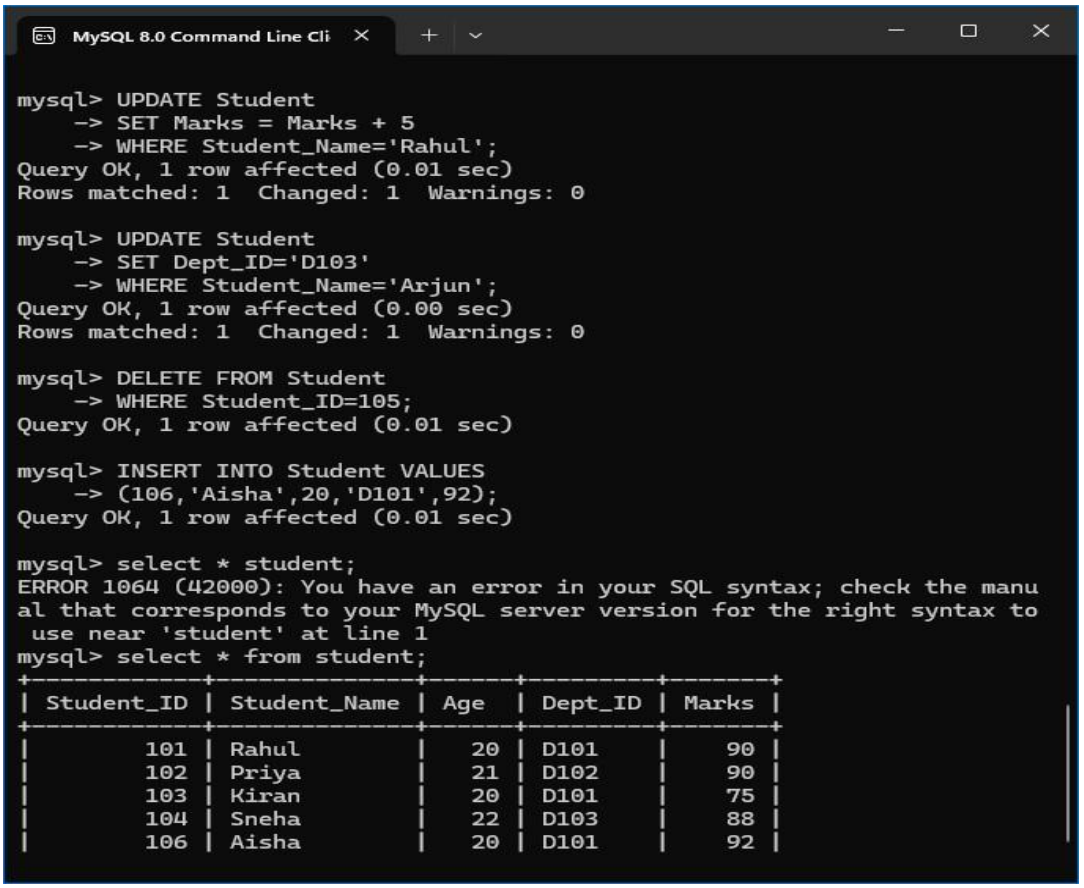


Retrieve students scoring above 80, Retrieve average marks department-wise

```
mysql> SELECT Student_Name, Marks
    -> FROM Student
    -> WHERE Marks > 80;
+--------------+-------+
| Student_Name | Marks |
+--------------+-------+
| Rahul        |    85 |
| Priya        |    90 |
| Sneha        |    88 |
+--------------+-------+
3 rows in set (0.00 sec)

mysql> SELECT Dept_ID,
    -> AVG(Marks) AS Average_Marks
    -> FROM Student
    -> GROUP BY Dept_ID;
+---------+---------------+
| Dept_ID | Average_Marks |
+---------+---------------+
| D101    |       80.0000 |
| D102    |       80.0000 |
| D103    |       88.0000 |
+---------+---------------+
3 rows in set (0.00 sec)
```

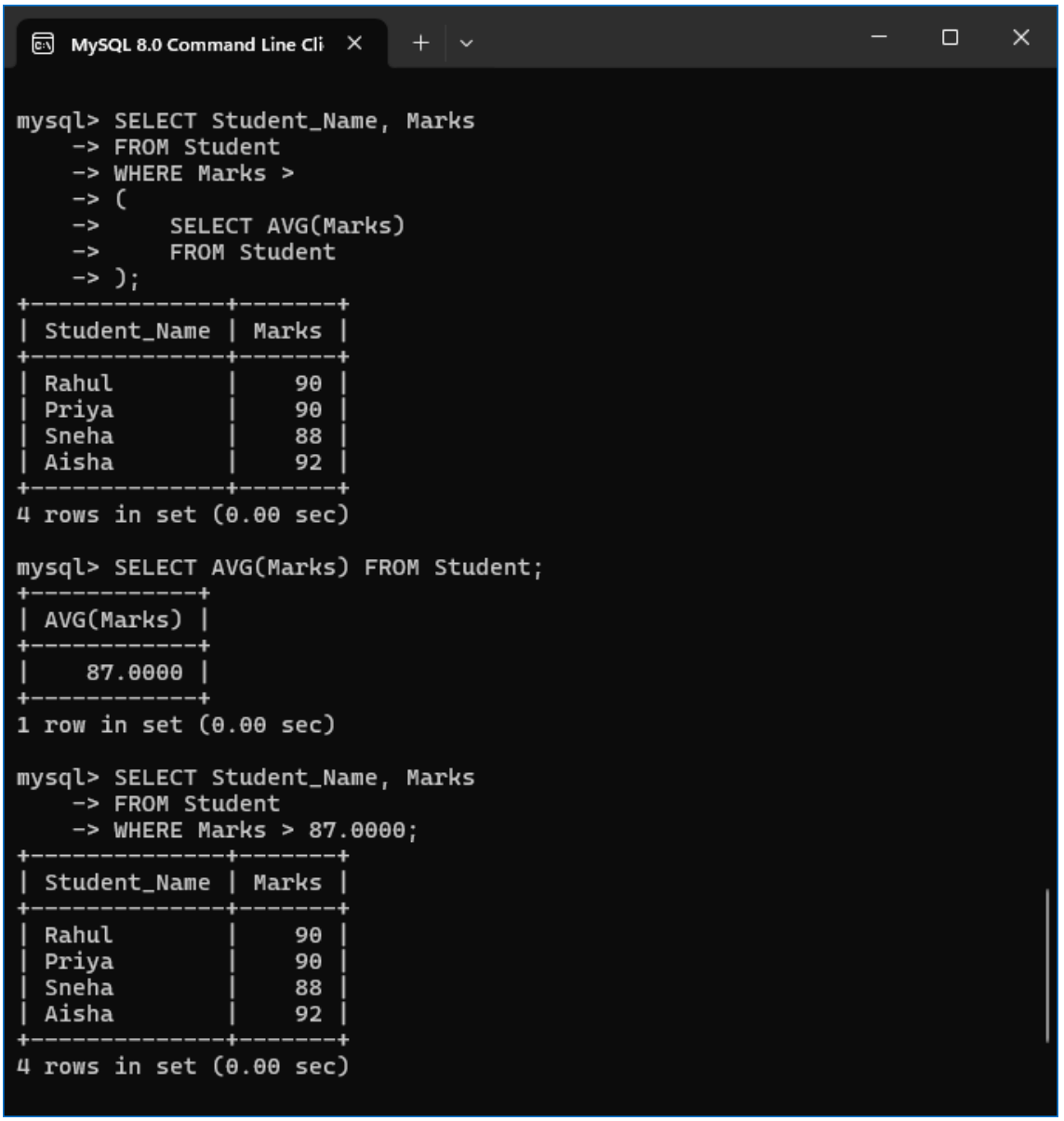
Update Queries
Increase Rahul's marks by 5, Change Arjun's department to Mechanical, Delete a student, add a student



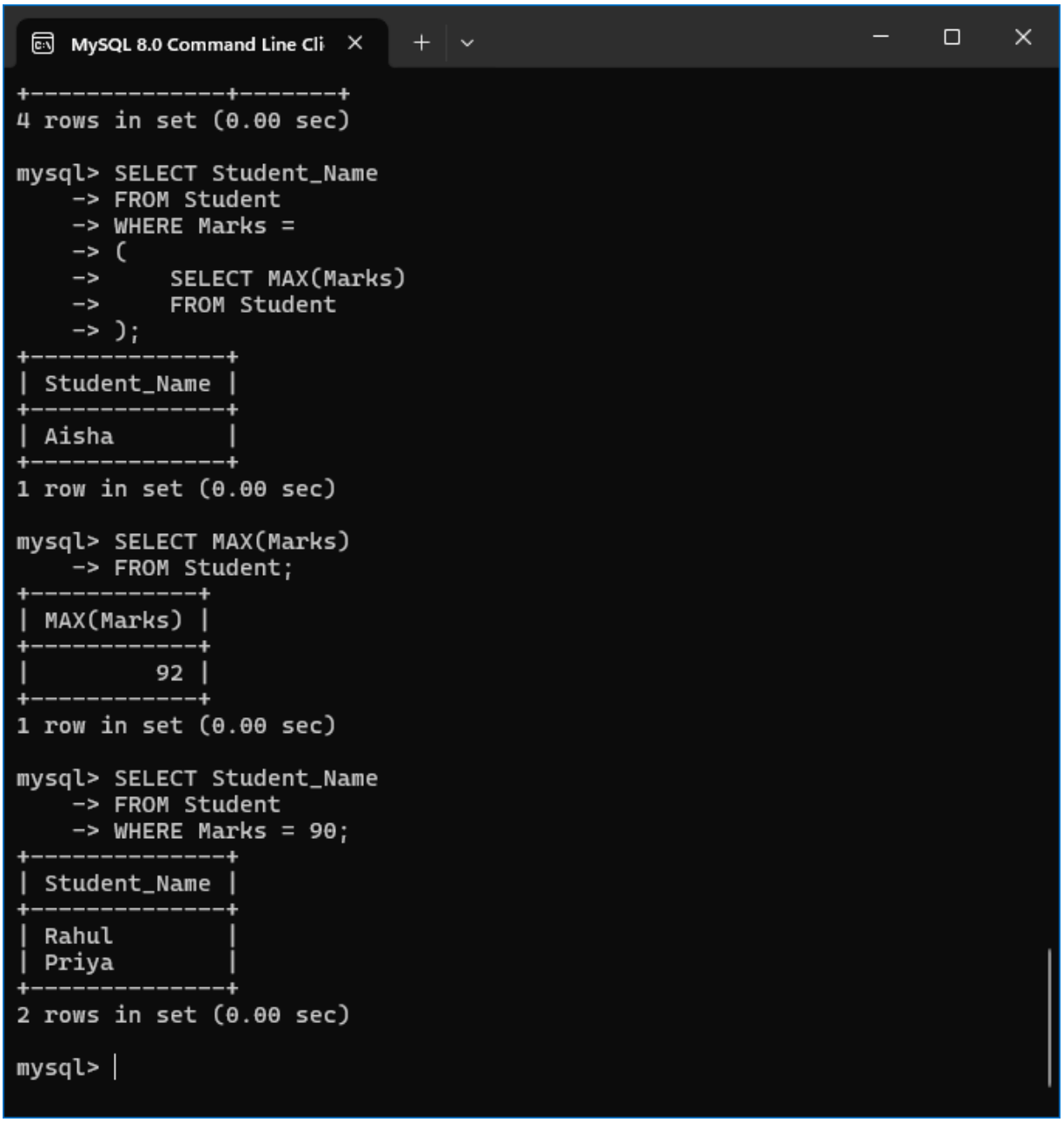
Justification:
A well-designed relational schema reduces data redundancy and maintains data integrity using primary and foreign keys. SQL queries enable efficient retrieval, insertion, updating, and deletion of data to meet application requirements.

4) Analyse a given SQL query with nested subqueries and predict its output, explaining each step of execution.

Nested Subquery with AVG()



Nested Subquery with MAX()



Justification:
Nested subqueries execute the inner query first and use its result in the outer query. This technique simplifies solving complex filtering and comparison problems within a single SQL statement.

5) Compare different SQL approaches (joins vs subqueries) for solving the same problem and evaluate their efficiency

Problem Statement
Retrieve the names of students along with their department names.

Relations:

Student(Student_ID, Student_Name, Dept_ID, Marks)

Department(Dept_ID, Dept_Name)

Approach 1: Using JOIN
SQL Query
SQL

```
SELECT Student.Student_Name, Department.Dept_Name
FROM Student
JOIN Department
ON Student.Dept_ID = Department.Dept_ID;
```

Working

- Combines the Student and Department tables using the common Dept_ID.
- Returns student names together with their department names in a single operation.

Advantages

- Faster for combining multiple tables.
- Easy to read and maintain.
- Optimized by most database management systems.
- Recommended for large datasets.

Approach 2: Using a Subquery

SQL Query
SQL

```
SELECT Student_Name,
(
   SELECT Dept_Name
   FROM Department
   WHERE Department.Dept_ID = Student.Dept_ID
) AS Department_Name
FROM Student;
```

Working

- The outer query retrieves each student's name.
- For every student row, the inner query searches the corresponding department name.

Advantages

- Useful when retrieving a single related value.
- Can be easier to understand for simple lookups.

Disadvantages

- The subquery may execute once for every row in the outer query.
- Can be slower on large tables.

Comparison Table

Feature	JOIN	Subquery
Performance	Faster	Slower (for many rows)
Readability	Easy for multiple tables	Good for simple queries
Execution	Single combined operation	Inner query executed repeatedly
Large Datasets	Highly efficient	Less efficient
Recommended Use	Retrieving related data from multiple tables	Simple lookups or filtering

Evaluation:

JOIN

- Best choice for combining related tables.
- Performs efficiently because the database optimizer processes both tables together.

Subquery

- Best suited for simple lookups or when one query depends on the result of another.
- May have additional overhead due to repeated execution.

Justification:
Joins are generally more efficient for retrieving data from multiple related tables, while subqueries are suitable when one query depends on the result of another. The choice depends on performance requirements and query complexity.

6) Design and implement views for a database system to provide restricted data access, and justify their use for security and abstraction

Scenario: Student Management System
Department Table, student table

```
MySQL 8.0 Command Line Cli

mysql> use co2;
Database changed
mysql> CREATE TABLE Department (
    ->     Dept_ID VARCHAR(5) PRIMARY KEY,
    ->     Dept_Name VARCHAR(30)
    -> );
Query OK, 0 rows affected (0.03 sec)

mysql> CREATE TABLE Student (
    ->     Student_ID INT PRIMARY KEY,
    ->     Student_Name VARCHAR(30),
    ->     Age INT,
    ->     Dept_ID VARCHAR(5),
    ->     Marks INT,
    ->     FOREIGN KEY (Dept_ID) REFERENCES Department(Dept_ID)
    -> );
Query OK, 0 rows affected (0.04 sec)

mysql> INSERT INTO Department VALUES
    -> ('D101','CSE'),
    -> ('D102','ECE'),
    -> ('D103','Mechanical');
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql>
mysql> INSERT INTO Student VALUES
    -> (101,'Rahul',20,'D101',85),
    -> (102,'Priya',21,'D102',90),
    -> (103,'Kiran',20,'D101',75),
    -> (104,'Sneha',22,'D103',88);
Query OK, 4 rows affected (0.01 sec)
Records: 4  Duplicates: 0  Warnings: 0
```

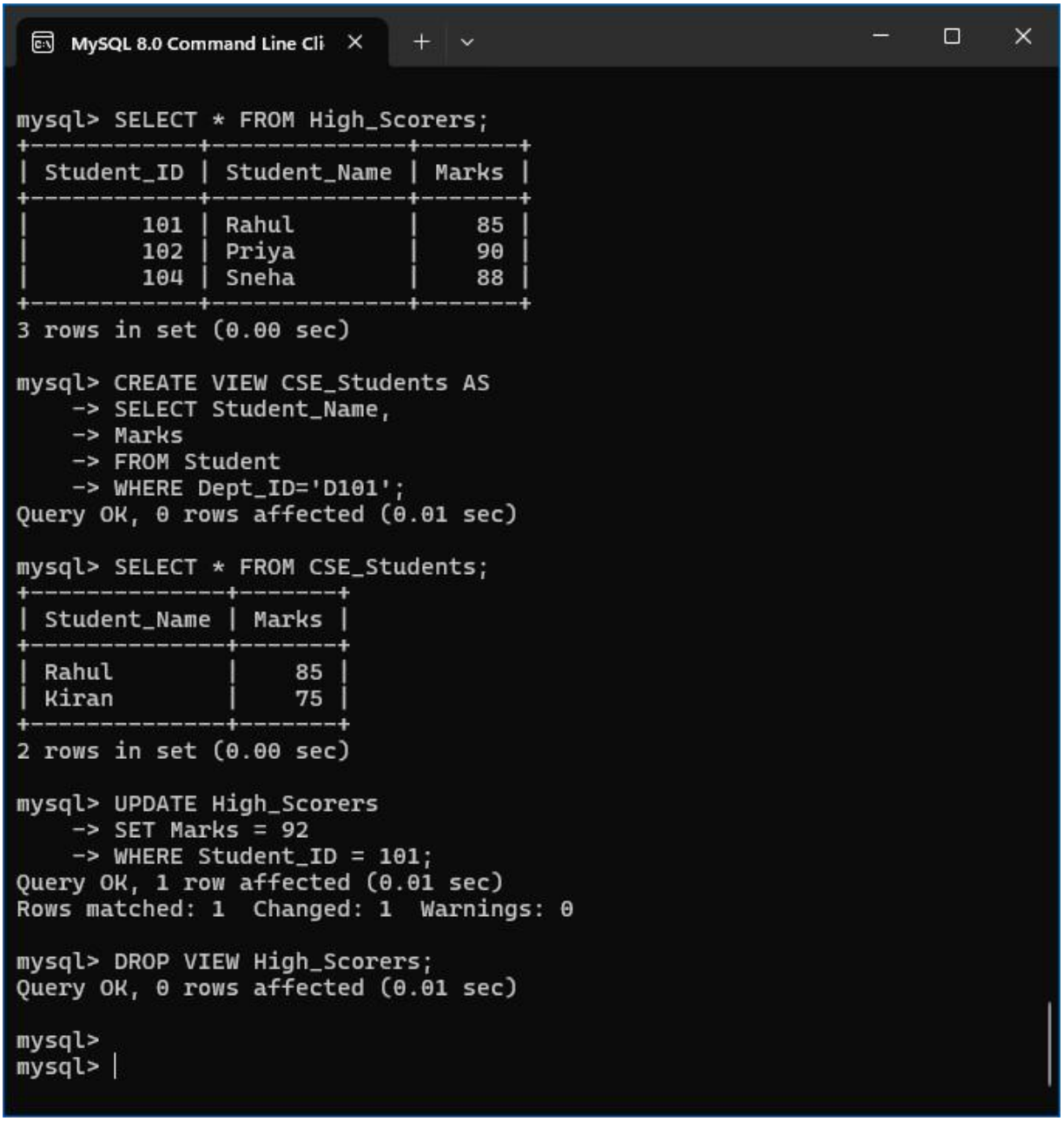
View 1: Student Details View

```
MySQL 8.0 Command Line Cli

mysql> CREATE VIEW Student_Details AS
    -> SELECT Student.Student_Name,
    -> Department.Dept_Name
    -> FROM Student
    -> JOIN Department
    -> ON Student.Dept_ID = Department.Dept_ID;
Query OK, 0 rows affected (0.01 sec)

mysql> SELECT * FROM Student_Details;
+--------------+------------+
| Student_Name | Dept_Name  |
+--------------+------------+
| Rahul        | CSE        |
| Kiran        | CSE        |
| Priya        | ECE        |
| Sneha        | Mechanical |
+--------------+------------+
4 rows in set (0.00 sec)
```

View 2: High Scorers View and View 3 : CSE _students



Justification:
Views provide restricted access by exposing only the required data while hiding sensitive information. They also simplify complex queries and improve security through data abstraction.

7) Given a database schema, write relational algebra expressions and convert them into equivalent SQL queries, analyzing differences in representation.

Given Database Schema
Student

Student_ID	Student_Name	Dept_ID	Marks

Department

Dept_ID	Dept_Name

Example 1: Students in the CSE Department

Relational Algebra

$$\pi_{Student\_Name,Dept\_Name}(\sigma_{Dept\_Name='CSE'}(Student \bowtie Department))$$

Equivalent SQL
SQL

```
SELECT Student.Student_Name, Department.Dept_Name
FROM Student
JOIN Department
ON Student.Dept_ID = Department.Dept_ID
WHERE Department.Dept_Name = 'CSE';
```

Analysis

- Relational Algebra: Uses σ (Selection), ⋈ (Join), and π (Projection).
- SQL: Uses WHERE, JOIN, and SELECT.
- Both return the same result, but SQL is more user-friendly and executable.

Example 2: Students Scoring More Than 80

Relational Algebra

$$\pi_{Student\_Name,Marks}(\sigma_{Marks>80}(Student))$$

Equivalent SQL

```
SELECT Student_Name, Marks
FROM Student
WHERE Marks > 80;
```

Analysis

- RA clearly separates selection and projection.
- SQL combines both operations into one query using SELECT and WHERE.

Example 3: Average Marks Department-wise

Relational Algebra (Extended)

$$\gamma_{Dept\_ID,\ AVG(Marks)\rightarrow Avg\_Marks}(Student)$$

Equivalent SQL :

```
SELECT Dept_ID,
AVG(Marks) AS Avg_Marks
FROM Student
GROUP BY Dept_ID;
```

Analysis

- Extended RA uses the γ (Grouping) operator.
- SQL uses GROUP BY with aggregate functions.
- SQL is easier to read and directly supported by DBMSs.

Example 4: Student Names with Department Names

Relational Algebra

$$\pi_{Student\_Name,Dept\_Name}(Student \bowtie Department)$$

Equivalent SQL:

```
SELECT Student.Student_Name,
Department.Dept_Name
FROM Student
JOIN Department
ON Student.Dept_ID = Department.Dept_ID;
```

Analysis

- RA uses the join operator ⋈.
- SQL uses the JOIN ... ON clause.
- Both produce the same output.

Comparison: Relational Algebra vs SQL

Relational Algebra	SQL
Procedural query language	Declarative query language
Uses symbols like σ, π, ⋈	Uses keywords like SELECT, FROM, WHERE
Describes the sequence of operations	Describes the required result
Mainly used for query design and optimization	Used to interact with real databases
Theoretical foundation	Practical implementation

Justification:
Relational algebra expresses database operations in a mathematical form, whereas SQL provides an executable implementation of those operations. Converting between them helps understand the logical execution of queries.

8) Optimize a set of inefficient SQL queries by restructuring them using joins, indexing concepts, or views, and evaluate performance improvements

Given Database Schema

Student(Student_ID, Student_Name, Dept_ID, Marks)
Department(Dept_ID, Dept_Name)

Example 1: Inefficient Query , Optimized query

```
mysql>
mysql> SELECT Student_Name
    -> FROM Student
    -> WHERE Dept_ID =
    -> (
    ->     SELECT Dept_ID
    ->     FROM Department
    ->     WHERE Dept_Name='CSE'
    -> );
+--------------+
| Student_Name |
+--------------+
| Rahul        |
| Kiran        |
+--------------+
2 rows in set (0.01 sec)

mysql> SELECT Student.Student_Name
    -> FROM Student
    -> JOIN Department
    -> ON Student.Dept_ID = Department.Dept_ID
    -> WHERE Department.Dept_Name = 'CSE';SELECT Student.Student_Name
+--------------+
| Student_Name |
+--------------+
| Rahul        |
| Kiran        |
+--------------+
2 rows in set (0.00 sec)
```

Example 2: Optimize by Adding an Index – Original Query , Creating an Index

```
2 rows in set (0.00 sec)

mysql> SELECT *
    -> FROM Student
    -> WHERE Student_ID = 101;
+------------+--------------+------+---------+-------+
| Student_ID | Student_Name | Age  | Dept_ID | Marks |
+------------+--------------+------+---------+-------+
|        101 | Rahul        |   20 | D101    |    92 |
+------------+--------------+------+---------+-------+
1 row in set (0.00 sec)

mysql> CREATE INDEX idx_studentid
    -> ON Student(Student_ID);
Query OK, 0 rows affected (0.09 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql>
```

Example 3: Optimize Repeated JOINs Using a View - Original Query, Create a View, Retrieve Data from the View

```
MySQL 8.0 Command Line Cli
Records: 0  Duplicates: 0  Warnings: 0

mysql> SELECT Student.Student_Name,
    -> Department.Dept_Name
    -> FROM Student
    -> JOIN Department
    -> ON Student.Dept_ID = Department.Dept_ID;
+--------------+------------+
| Student_Name | Dept_Name  |
+--------------+------------+
| Rahul        | CSE        |
| Kiran        | CSE        |
| Priya        | ECE        |
| Sneha        | Mechanical |
+--------------+------------+
4 rows in set (0.00 sec)

mysql> CREATE VIEW Student_Department AS
    -> SELECT Student.Student_Name,
    -> Department.Dept_Name,
    -> Student.Marks
    -> FROM Student
    -> JOIN Department
    -> ON Student.Dept_ID = Department.Dept_ID;
Query OK, 0 rows affected (0.01 sec)

mysql> SELECT * FROM Student_Department;
+--------------+------------+-------+
| Student_Name | Dept_Name  | Marks |
+--------------+------------+-------+
| Rahul        | CSE        |    92 |
| Kiran        | CSE        |    75 |
| Priya        | ECE        |    90 |
| Sneha        | Mechanical |    88 |
+--------------+------------+-------+
4 rows in set (0.00 sec)

mysql>
```

Example 4: Avoid SELECT *

```
mysql>
mysql> SELECT *
    -> FROM Student;
+------------+--------------+------+---------+-------+
| Student_ID | Student_Name | Age  | Dept_ID | Marks |
+------------+--------------+------+---------+-------+
|        101 | Rahul        |   20 | D101    |    92 |
|        102 | Priya        |   21 | D102    |    90 |
|        103 | Kiran        |   20 | D101    |    75 |
|        104 | Sneha        |   22 | D103    |    88 |
+------------+--------------+------+---------+-------+
4 rows in set (0.00 sec)

mysql> SELECT Student_Name, Marks
    -> FROM Student;
+--------------+-------+
| Student_Name | Marks |
+--------------+-------+
| Rahul        |    92 |
| Priya        |    90 |
| Kiran        |    75 |
| Sneha        |    88 |
+--------------+-------+
4 rows in set (0.00 sec)

mysql>
```

Optimization Method	Before	After	Benefit
JOIN	Subquery	JOIN	Faster table retrieval
Index	Full table scan	Indexed search	Faster searching
View	Repeated JOIN	View	Simpler and reusable queries
SELECT	`SELECT *`	Required columns only	Less data transferred

Justification:
Query optimization improves execution speed by using efficient joins, indexes, and appropriate query structures. Optimized queries reduce execution time, resource usage, and improve overall database performance.

9) Develop a complex SQL query involving grouping, aggregation, and nested queries to solve a real-world problem

Problem Statement
Find the departments whose average student marks are greater than the overall average marks of all students. Also display the department name, average marks, and number of students.

SQL Query
SQL

```
SELECT D.Dept_Name,
      COUNT(S.Student_ID) AS Total_Students,
      AVG(S.Marks) AS Average_Marks
FROM Student S
JOIN Department D
ON S.Dept_ID = D.Dept_ID
GROUP BY D.Dept_Name
HAVING AVG(S.Marks) >
(
   SELECT AVG(Marks)
   FROM Student
);
```

Explanation

Step 1: JOIN

SQL

Student JOIN Department
Combines both tables using Dept_ID.

Step 2: GROUP BY

SQL

GROUP BY D.Dept_Name
Groups students according to their departments.

Step 3: Aggregate Functions

SQL

COUNT(Student_ID)
AVG(Marks)
Counts students in each department.

Calculates the average marks of each department.

Step 4: Nested Query

SQL

SELECT AVG(Marks)
FROM Student;
Calculates the overall average marks of all students.

Step 5: HAVING Clause

SQL

```
HAVING AVG(S.Marks) >
(
   SELECT AVG(Marks)
   FROM Student
)
```

Displays only departments whose average marks exceed the overall average.

```
mysql> SELECT D.Dept_Name,
    ->        COUNT(S.Student_ID) AS Total_Students,
    ->        AVG(S.Marks) AS Average_Marks
    -> FROM Student S
    -> JOIN Department D
    -> ON S.Dept_ID = D.Dept_ID
    -> GROUP BY D.Dept_Name
    -> HAVING AVG(S.Marks) >
    -> (
    ->     SELECT AVG(Marks)
    ->     FROM Student
    -> );
+------------+----------------+---------------+
| Dept_Name  | Total_Students | Average_Marks |
+------------+----------------+---------------+
| ECE        |              1 |       90.0000 |
| Mechanical |              1 |       88.0000 |
+------------+----------------+---------------+
2 rows in set (0.00 sec)
```

MySQL 8.0 Command Line Cli

```
mysql> SELECT *
    -> FROM Student S
    -> INNER JOIN Department D
    -> ON S.Dept_ID = D.Dept_ID;
+------------+--------------+------+---------+-------+---------+--------
----+
| Student_ID | Student_Name | Age  | Dept_ID | Marks | Dept_ID | Dept_Na
me  |
+------------+--------------+------+---------+-------+---------+--------
----+
|        101 | Rahul        |   20 | D101    |    92 | D101    | CSE
    |
|        103 | Kiran        |   20 | D101    |    75 | D101    | CSE
    |
|        102 | Priya        |   21 | D102    |    90 | D102    | ECE
    |
|        104 | Sneha        |   22 | D103    |    88 | D103    | Mechani
cal |
+------------+--------------+------+---------+-------+---------+--------
----+
4 rows in set (0.00 sec)

mysql> SELECT D.Dept_Name,
    ->        COUNT(S.Student_ID) AS Total_Students,
    ->        AVG(S.Marks) AS Average_Marks
    -> FROM Student S
    -> INNER JOIN Department D
    -> ON S.Dept_ID = D.Dept_ID
    -> GROUP BY D.Dept_Name;
+------------+----------------+---------------+
| Dept_Name  | Total_Students | Average_Marks |
+------------+----------------+---------------+
| CSE        |              2 |       83.5000 |
| ECE        |              1 |       90.0000 |
| Mechanical |              1 |       88.0000 |
+------------+----------------+---------------+
3 rows in set (0.00 sec)
```

```
mysql> SELECT D.Dept_Name,
    ->        COUNT(S.Student_ID) AS Total_Students,
    ->        AVG(S.Marks) AS Average_Marks
    -> FROM Student S
    -> INNER JOIN Department D
    -> ON S.Dept_ID = D.Dept_ID
    -> GROUP BY D.Dept_Name;
+------------+----------------+---------------+
| Dept_Name  | Total_Students | Average_Marks |
+------------+----------------+---------------+
| CSE        |              2 |       83.5000 |
| ECE        |              1 |       90.0000 |
| Mechanical |              1 |       88.0000 |
+------------+----------------+---------------+
3 rows in set (0.00 sec)

mysql> SELECT AVG(Marks) AS Overall_Average
    -> FROM Student;
+-----------------+
| Overall_Average |
+-----------------+
|         86.2500 |
+-----------------+
1 row in set (0.00 sec)
```

OUTPUT

```
mysql> SELECT D.Dept_Name,
    ->        COUNT(S.Student_ID) AS Total_Students,
    ->        AVG(S.Marks) AS Average_Marks
    -> FROM Student S
    -> INNER JOIN Department D
    -> ON S.Dept_ID = D.Dept_ID
    -> GROUP BY D.Dept_Name
    -> HAVING AVG(S.Marks) >
    -> (
    ->     SELECT AVG(Marks)
    ->     FROM Student
    -> );
+------------+----------------+---------------+
| Dept_Name  | Total_Students | Average_Marks |
+------------+----------------+---------------+
| ECE        |              1 |       90.0000 |
| Mechanical |              1 |       88.0000 |
+------------+----------------+---------------+
2 rows in set (0.00 sec)
```

Justification:
Grouping and aggregate functions summarize data, while nested queries enable advanced filtering based on computed values. Combining these features helps solve real-world business problems effectively.

Q10. Given multiple relations, analyze the query requirements and decide whether to use Views, JOINs, or Subqueries, justifying your choice with reasoning.
Bloom's Level: BL5 – Evaluate
Relations
STUDENT(RegNo, Name, Gender, DeptNo, City)
DEPARTMENT(DeptNo, DeptName, HOD)
1. Using JOIN
Query
Display the student name along with the department name.

```
mysql> SELECT s.RegNo, s.Name, d.DeptName
    -> FROM Student s
    -> INNER JOIN Department d
    -> ON s.DeptNo = d.DeptNo;
+----------+--------+----------+
| RegNo    | Name   | DeptName |
+----------+--------+----------+
| 19221151 | Mohan  | CSE      |
| 19221153 | Ramesh | CSE      |
| 19221152 | Raju   | ECE      |
| 19221155 | San    | ECE      |
| 19221154 | Ram    | EEE      |
+----------+--------+----------+
5 rows in set (0.00 sec)
```

Reason

- **JOIN is used because data is required from both Student and Department tables.**
- **It is faster and more efficient for retrieving related data from multiple tables.**

- **2. Using SUBQUERY**
- **Query**
- **Display the names of students who belong to the CSE department.**

```
mysql> SELECT Name
    -> FROM Student
    -> WHERE DeptNo =
    -> (
    -> SELECT DeptNo
    -> FROM Department
    -> WHERE DeptName = 'CSE'
    -> );
+--------+
| Name   |
+--------+
| Mohan  |
| Ramesh |
+--------+
2 rows in set (0.05 sec)
```

Justification:
The appropriate technique depends on the query requirement: joins for combining related tables, subqueries for dependent calculations, and views for reusable, secure access. Choosing the right approach improves performance, readability, and maintainability.